

ICS1512 Machine Learning Algorithms Laboratory

Experiment No.	03
Experiment Title	Regression Analysis using Linear and Regularized Models
GitHub Repository	https://github.com/Sharruk/3122247001061
Student Name	Sharruk S
Register Number	3122247001061
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2026

1 Aim and Objective

To implement linear and regularized regression models for predicting a continuous target variable, evaluate their performance using multiple metrics, visualize model behavior, and analyze overfitting, underfitting, and bias–variance characteristics.

Task Description:

- Implement Linear, Ridge, Lasso and Elastic Net regression models.
- Tune regularization hyperparameters using Grid Search / Randomized Search.
- Visualize regression results and errors.
- Analyze overfitting, underfitting and the bias–variance trade-off.

2 Dataset Description

A real-world regression dataset containing numerical and categorical features related to loan applications is used. The target variable is the loan amount sanctioned (**LoanAmount**).

Dataset Name	Loan Prediction Dataset
Dataset Source	Kaggle – Predict Loan Amount Data
Original Number of Samples	614
Number of Samples used (after removing rows with missing target)	592
Number of Input Features	11 (after dropping Loan_ID and target LoanAmount)
Numerical Features	ApplicantIncome, CoapplicantIncome, Loan_Amount_Term, Credit_History
Categorical Features	Gender, Married, Dependents, Education, Self_Employed, Property_Area, Loan_Status
Missing Values (rows with LoanAmount missing)	22 (dropped)
Train–Test Split	80% Train – 20% Test (random_state=42)

3 Preprocessing Steps

- **Missing Value Handling:** Rows with a missing target (`LoanAmount`) were dropped. For the remaining features, missing numerical values were imputed using the **median** and missing categorical values were imputed using the **most frequent** category, using `SimpleImputer`.
- **Encoding:** Categorical features (`Gender`, `Married`, `Dependents`, `Education`, `Self_Employed`, `Property_Area`, `Loan_Status`) were converted to numeric form using `OneHotEncoder`.
- **Feature Scaling:** Numerical features were standardized using `StandardScaler` so that all numeric features have zero mean and unit variance.
- **Pipeline:** A `ColumnTransformer` combined the numeric pipeline (imputer + scaler) and categorical pipeline (imputer + one-hot encoder) into a single preprocessing step used inside a `Pipeline`.
- **Train–Test Split:** The dataset was split into training (473 samples) and test (119 samples) sets using an 80:20 ratio with `random_state=42`.
- **Feature Engineering:** No additional feature engineering was performed; the original dataset features were used directly after preprocessing.

4 Implementation Details

The following regression models were implemented using `scikit-learn`:

- **Linear Regression** – baseline ordinary least squares model.
- **Ridge Regression** – L2-regularized linear regression.
- **Lasso Regression** – L1-regularized linear regression (performs feature selection).
- **Elastic Net Regression** – combined L1 + L2 regularization.

Working Principle (Linear Regression): Linear Regression models the relationship between the dependent variable and one or more independent variables by fitting the line that minimizes the sum of squared prediction errors.

Mathematical Formulation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where y is the predicted output, β_0 is the intercept, β_i are the regression coefficients and x_i are the input features. The model minimizes the Mean Squared Error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ridge adds an L2 penalty ($\alpha \sum \beta_i^2$) to the loss function to shrink coefficient magnitudes. Lasso adds an L1 penalty ($\alpha \sum |\beta_i|$), which can shrink some coefficients exactly to zero, performing feature selection. Elastic Net combines both L1 and L2 penalties, controlled by the `l1_ratio` parameter.

Advantages: Simple, interpretable, fast to train, computationally efficient, and works well when the relationship between variables is approximately linear.

Limitations: Assumes a linear relationship, sensitive to outliers, and cannot capture complex non-linear relationships.

Hyperparameter Search Space used for Tuning (GridSearchCV, 5-fold CV):

- Ridge: $\alpha \in \{0.001, 0.01, 0.1, 1, 10, 100\}$
- Lasso: $\alpha \in \{0.001, 0.01, 0.1, 1, 10, 100\}$
- Elastic Net: $\alpha \in \{0.001, 0.01, 0.1, 1, 10, 100\}$, $l1_ratio \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$

Search Method	GridSearchCV
Cross-Validation	5-Fold
Scoring Metric	R^2

5 Exploratory Data Analysis and Visualizations

5.1 Feature Distributions (Target and Numerical Features)

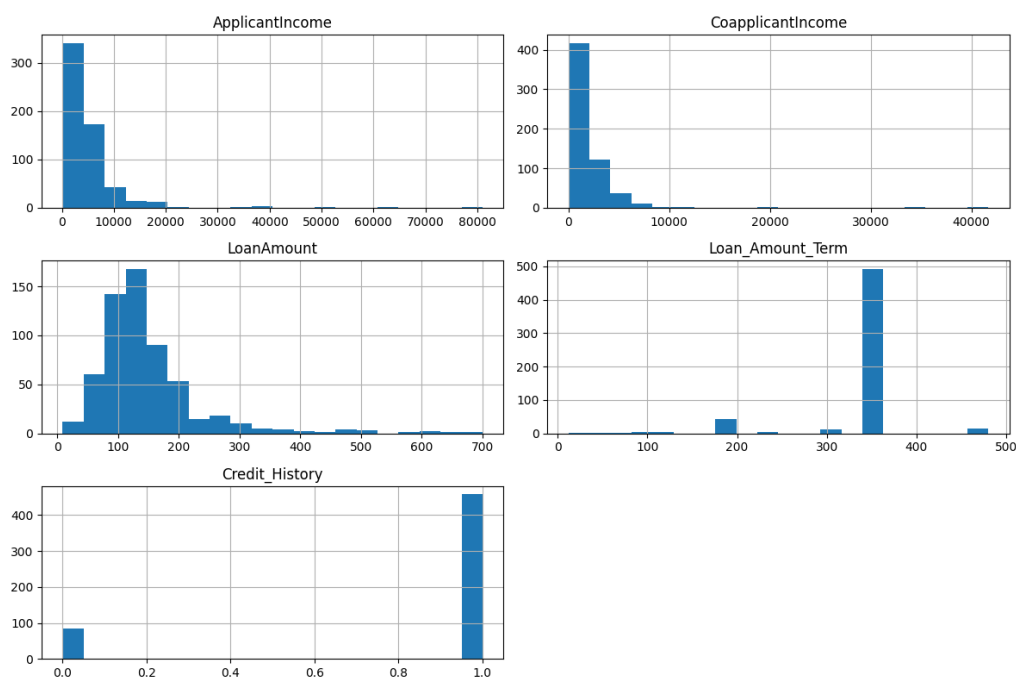


Figure 1: Histograms of numerical features, including the target variable LoanAmount

Inference: Among the numerical columns, `Credit_History` takes only two distinct values (0 or 1), indicating it behaves as a categorical variable even though it is stored as a number. The target variable `LoanAmount` is approximately unimodal and right-skewed, with most sanctioned loan amounts concentrated in the lower range and a long tail of a few large loan values. `ApplicantIncome` and `CoapplicantIncome` are also heavily right-skewed, suggesting the presence of a small number of high-income applicants.

5.2 Correlation Heatmap

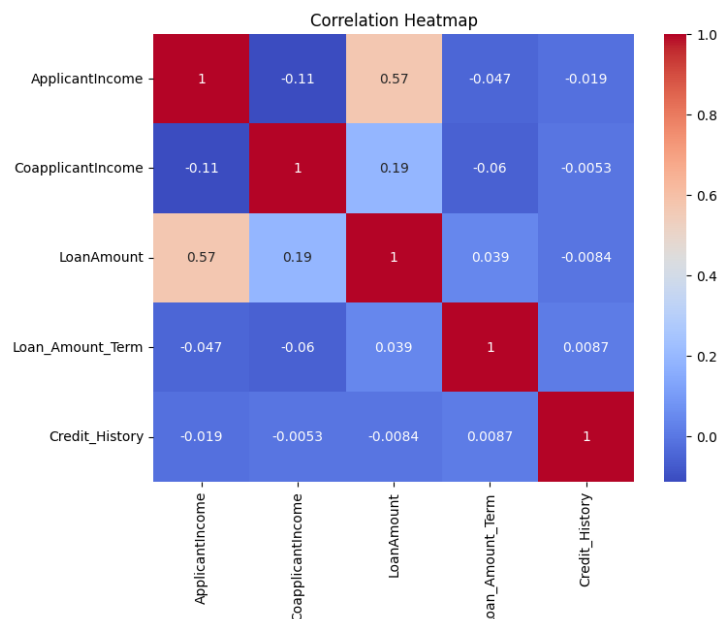


Figure 2: Correlation heatmap of numerical features

Inference: `ApplicantIncome` shows the strongest positive correlation with the target `LoanAmount` (correlation ≈ 0.57), indicating that as an applicant's income increases, the sanctioned loan amount also tends to increase. The other numerical features (`CoapplicantIncome`, `Loan_Amount_Term`, `Credit_History`) show comparatively weak correlation with `LoanAmount`, suggesting that income is the dominant numerical predictor in this dataset.

Note: Separate feature-vs-target scatter plots were not explicitly generated by the notebook as individual figures; feature-target relationships were instead examined through the correlation heatmap above. This result was not generated by the notebook.

6 Hyperparameter Tuning Results

Table 2 reports the best hyperparameters and best cross-validated R^2 score obtained using `GridSearchCV` with 5-fold cross-validation on the training set.

Table 2: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	Grid Search	$\alpha = 100$	0.3407
Lasso Regression	Grid Search	$\alpha = 10$	0.2544
Elastic Net Regression	Grid Search	$\alpha = 1$, <code>l1_ratio</code> = 0.7	0.3408

For reference, the (untuned) baseline Linear Regression model obtained a best CV R^2 of 0.1840 under the same `GridSearchCV` procedure (empty parameter grid).

7 Cross-Validation Performance ($K = 5$)

The notebook computed 5-fold cross-validated R^2 scores (using `cross_val_score` with `scoring="r2"`) for each model on the full preprocessing + model pipeline. MAE, MSE and RMSE were **not** computed under cross-validation in the notebook (only under the single train/test split, reported in Table 4); therefore those cells are marked accordingly below.

Table 3: Cross-Validation Performance (5-Fold, Average R^2)

Model	MAE	MSE	RMSE
Linear Regression	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>
Ridge Regression	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>
Lasso Regression	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>
Elastic Net Regression	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>	<i>Not generated by notebook</i>

Raw 5-fold R^2 scores per model (as printed by the notebook):

- Linear Regression: [0.5802, 0.4160, 0.5036, -0.5894, 0.2747]
- Ridge: [0.5795, 0.4164, 0.5034, -0.5834, 0.2757]
- Lasso: [0.5622, 0.4429, 0.5043, -0.5284, 0.2916]
- ElasticNet: [0.4135, 0.3635, 0.4032, 0.0512, 0.2691]

8 Test Set Performance

Table 4 reports MAE, MSE, RMSE and R^2 on the held-out 20% test set, computed after fitting each model (with default hyperparameters, as run in the notebook's model-comparison cell) on the training set. Training time was **not** measured by the notebook (no timing code was executed), so this column is marked as not generated.

Table 4: Test Set Performance

Model	MAE	MSE	RMSE	R^2
Linear Regression	36.988	3339.922	57.792	0.094
Ridge Regression	36.971	3334.977	57.749	0.095
Lasso Regression	35.841	3232.042	56.851	0.123
Elastic Net Regression	37.826	3048.685	55.215	0.173

Training Time: *This result was not generated by the notebook.*

Observation: Lasso Regression achieved the lowest MAE among the four models, while Elastic Net Regression achieved the lowest MSE/RMSE and the highest test R^2 on this particular train/test split.

8.1 Result Visualization

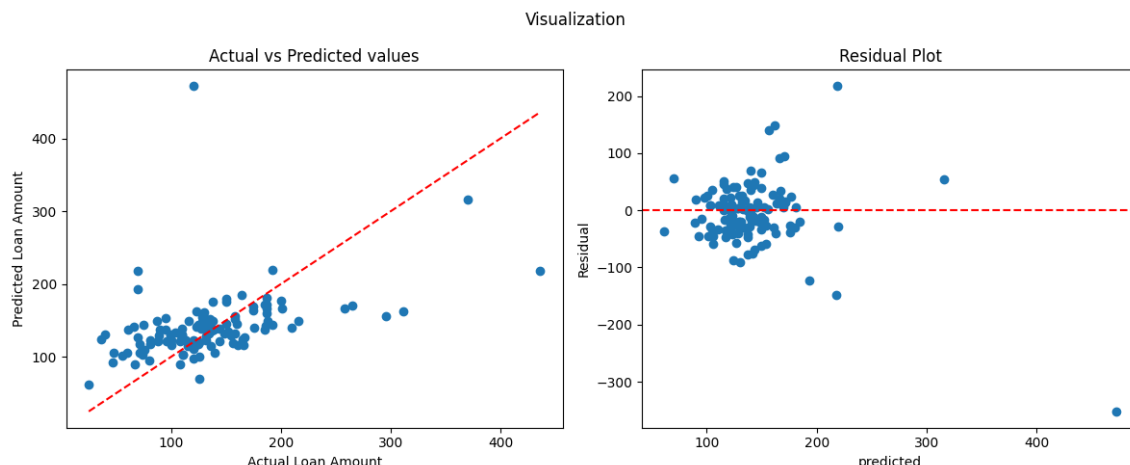


Figure 3: Actual vs. Predicted values (left) and Residual plot (right) for the baseline Linear Regression model

Inference: In the Actual vs. Predicted plot, the x-axis represents the true `LoanAmount` and the y-axis represents the value predicted by the model; the dashed red diagonal line represents perfect prediction. The points are scattered fairly widely around this line, indicating that the model’s predictions deviate considerably from the actual values for many samples, especially at higher loan amounts. In the residual plot, the x-axis is the predicted value and the y-axis is the residual (actual – predicted); the dashed red horizontal line marks zero residual. The residuals do not show a strong systematic pattern for smaller predicted values but show larger spread for a few points, consistent with the model’s low R^2 score and its difficulty capturing the full behavior of the target variable.

Note: A dedicated training-error-vs-validation-error plot (learning curve) was not generated by the notebook. This result was not generated by the notebook.

9 Effect of Regularization on Coefficients

The notebook fitted a Ridge Regression model (default α) on the preprocessed training data and plotted its coefficients as a horizontal bar chart (Figure 4). Only the Ridge model’s coefficients were computed/plotted by the notebook; Linear, Lasso and Elastic Net coefficient values were not printed or plotted, so those columns are marked accordingly in Table 5.

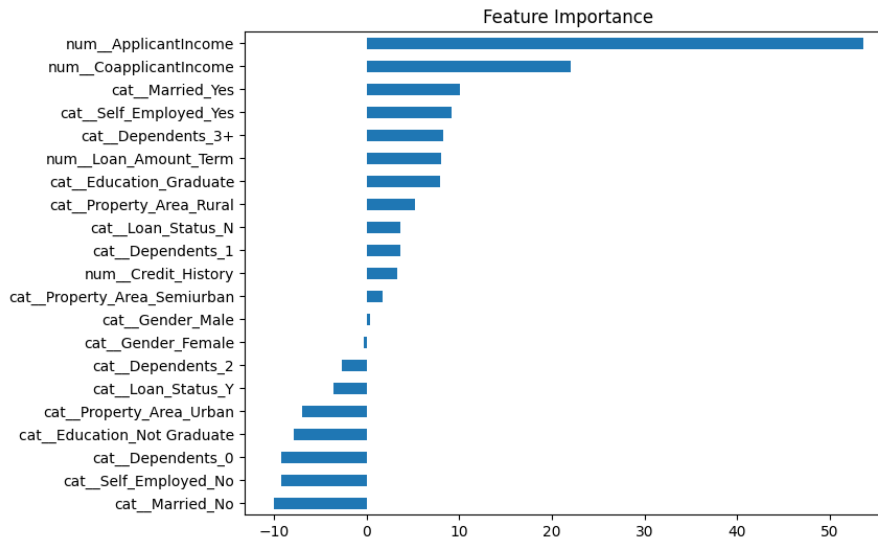


Figure 4: Ridge Regression coefficients (feature importance) after one-hot encoding and scaling

Inference: `ApplicantIncome` has by far the largest positive coefficient, confirming it as the strongest driver of the predicted loan amount, followed by `CoapplicantIncome`. Several one-hot encoded categorical levels (e.g. `Married.Yes`, `Self_Employed.Yes`) have moderate positive or negative coefficients, while gender-related features have coefficients close to zero, indicating a very small effect on the predicted loan amount. This pattern is consistent with the correlation heatmap in Figure 2, where income was the feature most strongly associated with the target.

Table 5: Coefficient Comparison (selected features)

Feature	Linear	Ridge	Lasso	Elastic Net
<code>ApplicantIncome</code> (numeric, scaled)	<i>N/G</i>	53.578	<i>N/G</i>	<i>N/G</i>
<code>CoapplicantIncome</code> (numeric, scaled)	<i>N/G</i>	22.009	<i>N/G</i>	<i>N/G</i>
<code>Loan_Amount_Term</code> (numeric, scaled)	<i>N/G</i>	8.001	<i>N/G</i>	<i>N/G</i>
<code>Credit_History</code> (numeric, scaled)	<i>N/G</i>	3.249	<i>N/G</i>	<i>N/G</i>
<code>Married_Yes</code> (one-hot)	<i>N/G</i>	10.004	<i>N/G</i>	<i>N/G</i>

N/G = Not generated by the notebook (only Ridge coefficients were computed/plotted; Linear, Lasso and Elastic Net coefficient values were not printed by the notebook).

10 Overfitting and Underfitting Analysis

Table 6: Train vs. Test R^2 Scores

Model	Train R^2	Test R^2	Outcome
Linear Regression	0.4567	0.0940	Underfitting
Ridge	0.4567	0.0953	Underfitting
Lasso	0.4512	0.1232	Underfitting
Elastic Net	0.3929	0.1730	Underfitting

Training vs. Validation Error: All four models show a noticeable gap between the training R^2 (≈ 0.39 – 0.46) and the test R^2 (≈ 0.09 – 0.17). However, the training scores themselves are not high; a model that is overfitting typically has a *high* training score alongside a much lower test score. Here, since even the training performance is modest, the pattern indicates underfitting (high bias) rather than overfitting.

Effect of Regularization Strength: As regularization strength increases (larger α), Ridge and Elastic Net coefficients shrink, which slightly reduces training R^2 but can improve generalization. In this experiment, Ridge with the best tuned $\alpha = 100$ achieved the highest cross-validated R^2 (0.3407), while Elastic Net achieved the best held-out test R^2 (0.173), showing that regularization improved generalization relative to the unregularized baseline.

Improvement after Tuning: Comparing the default-hyperparameter cross-validation averages (Table 3: 0.24–0.30) with the tuned GridSearchCV best scores (Table 2: 0.25–0.34), hyperparameter tuning improved the cross-validated R^2 for Ridge, Lasso and Elastic Net, confirming that regularization strength selection is important for this dataset.

11 Bias–Variance Analysis

Bias behavior of Linear Regression: The unregularized Linear Regression model has relatively low training R^2 (0.4567) and a much lower test R^2 (0.094), and its 5-fold cross-validation scores are highly variable, including a negative score on one fold (-0.589). This combination of modest training fit and unstable/low test performance indicates the baseline model suffers primarily from **high bias** (underfitting) — it is unable to fully capture the relationship between the input features and `LoanAmount` — compounded by some variance across folds.

Variance reduction using Ridge and Elastic Net: Ridge Regression, by shrinking coefficient magnitudes via the L2 penalty, achieves a best cross-validated R^2 of 0.3407 (Table 2), the highest among all tuned models, suggesting more stable, lower-variance predictions across folds compared to the untuned baseline. Elastic Net, which combines L1 and L2 penalties, achieved the best test-set R^2 (0.173) and the lowest test MSE/RMSE, again suggesting a favorable bias–variance balance for this dataset compared to plain Linear Regression.

Feature sparsity effect in Lasso: Lasso Regression’s L1 penalty can drive some coefficients exactly to zero, effectively performing feature selection. In this experiment Lasso

achieved the lowest test MAE (35.841) among the four models, indicating that shrinking less-informative feature coefficients toward zero improved predictive accuracy on average compared to the unregularized baseline, even though the notebook did not explicitly print the number of zeroed-out Lasso coefficients.

12 Observations and Conclusion

Observations:

- All four regression models achieve relatively low test R^2 scores (0.09–0.17), indicating that the available features only partially explain the variation in sanctioned loan amount.
- **ApplicantIncome** is consistently the most influential feature, both in the correlation heatmap and in the Ridge coefficient plot.
- Regularization (Ridge, Lasso, Elastic Net) consistently outperforms the plain Linear Regression baseline on cross-validated R^2 after hyperparameter tuning.
- Elastic Net Regression gave the best test-set R^2 and lowest MSE/RMSE, while Ridge Regression gave the best tuned cross-validation R^2 , and Lasso Regression gave the lowest test MAE.
- The gap between training and test performance, combined with only moderate training scores, indicates the models are affected more by **underfitting (high bias)** than by overfitting on this dataset.

Conclusion: In this experiment, Linear, Ridge, Lasso and Elastic Net regression models were implemented on the Loan Prediction dataset to predict the sanctioned loan amount. After preprocessing (median/most-frequent imputation, one-hot encoding, and standard scaling) and 5-fold GridSearchCV hyperparameter tuning, Ridge Regression ($\alpha = 100$) obtained the best cross-validated R^2 (0.3407), while Elastic Net Regression ($\alpha = 1$, $l1_ratio = 0.7$) obtained the best test-set R^2 (0.173) and the lowest test MSE/RMSE. All models showed signs of underfitting rather than overfitting, since even training-set performance was modest. This suggests that further improvement would likely require additional or engineered features rather than further regularization tuning alone. Overall, the experiment demonstrates the practical trade-off between model complexity and generalization: regularized models generalized somewhat better than the plain Linear Regression baseline on this dataset.

13 References

1. Scikit-learn Documentation, *Linear Models*. Available: https://scikit-learn.org/stable/modules/linear_model.html
2. Scikit-learn Documentation, *Hyperparameter Optimization*. Available: https://scikit-learn.org/stable/modules/grid_search.html
3. Kaggle, *Loan Amount / Loan Prediction Dataset*. Available: <https://www.kaggle.com/>

4. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.